Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ДОМАШНЯЯ РАБОТА № 2

«Разработка консольного приложения с применением ООП»

ДИСЦИПЛИНА: «Архитектура программных систем»

Выполнил: студент гр. ИУ5-21Б _____ (Ломаско Р. А.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю. Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):
- Балльная оценка:
- Оценка:

2026 г.

Цель: Освоить принципы объектно-ориентированного программирования при создании консольного приложения на C#: проектирование классов с инкапсуляцией и валидацией данных, работа с базой данных SQLite через класс-обёртку, применение паттерна Fluent Interface для построения отчётов.

Задачи:
1. *Создать классы-модели Airline и Flight с валидацией числового поля.*
2. *Реализовать класс DatabaseManager, инкапсулирующий работу с SQLite.*
3. *Разработать консольное меню с операциями CRUD для рейсов.*
4. *Реализовать класс ReportBuilder с паттерном Fluent Interface.*
5. *Сформировать три отчёта с помощью SQL-запросов и класса ReportBuilder.*
6. *Добавить в ReportBuilder метод Numbered() - нумерацию строк (задание группы А)*

Выполнение работы

1. Предметная область (вариант 14)

Вариант 14 задаёт следующую пару таблиц:

Таблица	Поля	Числовое поле	Описание
Авиакомпании (справочник)	`airline_id, airline_name`	—	Сторона «один»
Рейсы (основная)	`flight_id, airline_id, flight_name, distance_km`	`distance_km`	Дальность маршрута (км), сторона «много»

Таблица 1 – Структура данных варианта 14

2. CSV-файлы с тестовыми данными

Созданы два файла с разделителем «;»:

```
# airlines.csv
airline_id;airline_name
1;Аэрофлот
2;S7 Airlines
3;Победа
4;Уральские авиалинии
5;Utair
```

```
# flights.csv (14 записей, 2–4 рейса на авиакомпанию)
flight_id;airline_id;flight_name;distance_km
1;1;Москва — Санкт-Петербург;634
2;1;Москва — Сочи;1354
3;1;Москва — Владивосток;6430
4;1;Москва — Нью-Йорк;9754
5;2;Новосибирск — Москва;2812
6;2;Новосибирск — Санкт-Петербург;3194
7;2;Москва — Лондон;2504
8;3;Москва — Екатеринбург;1416
9;3;Москва — Казань;822
10;3;Москва — Краснодар;1185
11;4;Екатеринбург — Москва;1416
12;4;Екатеринбург — Сочи;2200
13;5;Тюмень — Москва;2144
14;5;Ханты-Мансийск — Москва;2410
```

3. Классы-модели

Для справочной таблицы создан класс Airline с двумя свойствами. Для основной таблицы — класс Flight, в котором поле DistanceKm имеет валидацию в сеттере: при отрицательном значении выбрасывается ArgumentException. Оба класса имеют два конструктора (с параметрами и по умолчанию через this(…)) и переопределяют ToString().

```
// Airline.cs
class Airline
{
    public int Id { get; set; }
    public string Name { get; set; }

    public Airline(int id, string name) { Id = id; Name = name; }
    public Airline() : this(0, «») { }

    public override string ToString() => $”[{Id}] {Name}”;
}
```

```
// Flight.cs
class Flight
{
    public int Id { get; set; }
    public int AirlineId { get; set; }
    public string Name { get; set; }

    private int _distanceKm;
    public int DistanceKm
    {
        get => _distanceKm;
        set
        {
            if (value < 0)
                throw new ArgumentException(«Дальность маршрута не может быть
отрицательной»);
            _distanceKm = value;
        }
    }

    public Flight(int id, int airlineId, string name, int distanceKm)
    { Id = id; AirlineId = airlineId; Name = name; DistanceKm = distanceKm; }

    public Flight() : this(0, 0, «», 0) { }

    public override string ToString()
        => $»[{Id}] {Name}, авиакомпания #{AirlineId}, дальность: {DistanceKm}
км»;
}
```

Листинг 1 – Файлы Airline.cs и Flight.cs

4. Класс DatabaseManager

Класс DatabaseManager инакпсулирует всю работу с SQLite. Для устранения дублирования кода используется вспомогательный метод Connect(), возвращающий открытое соединение. Таблицы создаются автоматически в конструкторе. Метод LoadFromCsv() импортирует данные из CSV-файлов при первом запуске, если таблицы пусты.
Класс предоставляет методы: GetAllAirlines(), GetAllFlights(), GetFlightById(), AddFlight(), UpdateFlight(), DeleteFlight(), а также ExecuteQuery() - для выполнения произвольных SQL-запросов из класса ReportBuilder.

```
// DatabaseManager.cs (ключевые части)
class DatabaseManager
{
    private string _dbPath;

    public DatabaseManager(string dbPath)
    {
        _dbPath = dbPath;
        CreateTables();   // таблицы создаются сразу при инициализации
    }

    private SqliteConnection Connect()
    {
        var conn = new SqliteConnection($"Data Source={_dbPath}");
        conn.Open();
        return conn;
    }

    public List<Flight> GetAllFlights()
    {
        var list = new List<Flight>();
        using var conn = Connect();
        var cmd = conn.CreateCommand();
        cmd.CommandText = "SELECT … FROM flight ORDER BY flight_id";
        using var r = cmd.ExecuteReader();
        while (r.Read())
            list.Add(new Flight(r.GetInt32(0), r.GetInt32(1), r.GetString(2),
r.GetInt32(3)));
        return list;
    }

    public (string[] columns, List<string[]> rows) ExecuteQuery(string sql)
    {
        using var conn = Connect();
        var cmd = conn.CreateCommand();
        cmd.CommandText = sql;
        using var r = cmd.ExecuteReader();
        // читаем имена столбцов и строки результата…
        return (columns, rows);
    }
}
```

Листинг 2 – Класс DatabaseManager (фрагмент)

5. Класс ReportBuilder (Fluent Interface)

Класс ReportBuilder реализует паттерн Fluent Interface: промежуточные методы Query(), Title(), Header(), ColumnWidths(), Numbered() сохраняют настройки и возвращают this, что позволяет записывать вызовы в цепочку. Терминальный метод Build() выполняет SQL через DatabaseManager и форматирует результат с помощью StringBuilder. Метод Print() вызывает Build() и выводит результат в консоль.

Задание группы А: добавлен метод Numbered(), который включает нумерацию строк. Если он вызван в цепочке, в начало каждой строки добавляется порядковый номер (столбец «№» шириной 5 символов).

```
// ReportBuilder.cs
class ReportBuilder
{
    private DatabaseManager _db;
    private string _sql = «», _title = «»;
    private string[] _headers = new string[0];
    private int[] _widths = new int[0];
    private bool _numbered = false;

    public ReportBuilder(DatabaseManager db) { _db = db; }

    // Промежуточные методы
    public ReportBuilder Query(string sql)                 { _sql = sql;
return this; }
    public ReportBuilder Title(string title)               { _title = title;
return this; }
    public ReportBuilder Header(params string[] cols)      { _headers = cols;
return this; }
    public ReportBuilder ColumnWidths(params int[] widths) { _widths = widths;
return this; }
    public ReportBuilder Numbered() { _numbered = true; return this; } // Группа A

    public string Build()
    {
        var (columns, rows) = _db.ExecuteQuery(_sql);
        var sb = new StringBuilder();
        string[] headers = _headers.Length > 0 ? _headers : columns;
        int n = headers.Length;
        int[] w = new int[n];
        for (int i = 0; i < n; i++) w[i] = i < _widths.Length ? _widths[i] : 20;
        int numW = _numbered ? 5 : 0;
        int total = numW; for (int i = 0; i < n; i++) total += w[i];
        string sep = new string('-', total);
        if (_title.Length > 0) sb.AppendLine($"\n=== {_title} ===");
        if (_numbered) sb.Append("№".PadRight(numW));
        for (int i = 0; i < n; i++) sb.Append(headers[i].PadRight(w[i]));
        sb.AppendLine(); sb.AppendLine(sep);
        for (int r = 0; r < rows.Count; r++)
        {
            if (_numbered) sb.Append((r+1).ToString().PadRight(numW));
            for (int c = 0; c < rows[r].Length && c < n; c++)
                sb.Append(rows[r][c].PadRight(w[c]));
            sb.AppendLine();
        }
        return sb.ToString();
    }
    public void Print() => Console.Write(Build());
}
```

Листинг 3 – Класс ReportBuilder

6. Консольное меню

Главный цикл программы выводит меню и обрабатывает выбор пользователя. При добавлении рейса сначла показывается список авиакомпаний для выбора внешнего ключа. При редактировании выводятся текущие значения, и пользователь может оставить поле без изменений, нажав Enter. Все операции обёрнуты в try/catch для обработки ошибок ввода.

```
// Program.cs — главный цикл
var db = new DatabaseManager("airlines.db");
db.LoadFromCsv(airlineCsv, flightCsv);

string choice;
do {
    Console.WriteLine(«1) Показать авиакомпании»);
    Console.WriteLine(«2) Показать рейсы»);
    Console.WriteLine(«3) Добавить рейс»);
    Console.WriteLine(«4) Редактировать рейс»);
    Console.WriteLine(«5) Удалить рейс»);
    Console.WriteLine(«6) Отчёты»);
    Console.WriteLine(«0) Выход»);
    choice = Console.ReadLine()?.Trim() ?? «»;
    switch (choice) { case "1": ShowAirlines(db); break; … }
} while (choice != «0»);

// Пример функции отчёта (отчёт 1 с нумерацией)
static void Report1_FlightsWithAirlines(DatabaseManager db)
{
    new ReportBuilder(db)
        .Query(@"SELECT f.flight_name, a.airline_name, f.distance_km
                 FROM flight f JOIN airline a ON f.airline_id = a.airline_id
                 ORDER BY f.flight_name")
        .Title(«Рейсы по авиакомпаниям»)
        .Header(«Маршрут», «Авиакомпания», «Дальность (км)»)
        .ColumnWidths(30, 22, 15)
        .Numbered()
        .Print();
}
```

Листинг 4 – Фрагмент Program.cs

7. Результаты работы программы

При первом запуске программа загружает данные из CSV-файлов и открывает главное меню:

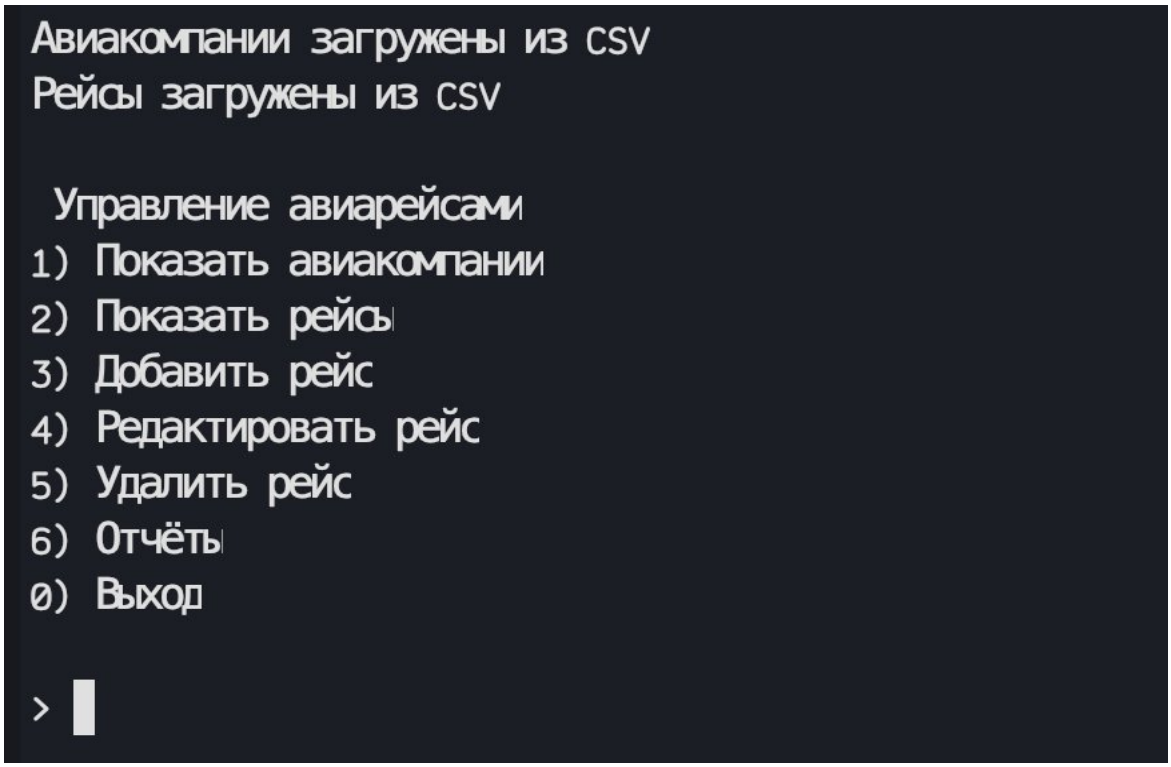

Рисунок 1 – Запуск программы и главное меню

Пример работы отчёта 1 «Рейсы по авиакомпаниям» с нумерацией строк (задание группы А):

```
=== Рейсы по авиакомпаниям ===
№   Маршрут                        Авиакомпания          Дальность (км)
----------------------------------------------------------------------
1   А - В                          Utair                 1
2   Екатеринбург — Москва          Уральские авиалинии   1416
3   Екатеринбург — Сочи            Уральские авиалинии   2200
4   Москва — Екатеринбург          Победа                1416
5   Москва — Казань                Победа                822
6   Москва — Краснодар             Победа                1185
7   Москва — Лондон                S7 Airlines           2504
8   Москва — Нью-Йорк              Аэрофлот              9754
9   Москва — Санкт-Петербург       Аэрофлот              634
10  Москва — Сочи                  Аэрофлот              1354
11  Новосибирск — Москва           S7 Airlines           2812
12  Новосибирск — Санкт-Петербург  S7 Airlines           3194
13  Тюмень — Москва                Utair                 2144
14  Ханты-Мансийск — Москва        Utair                 2410
```

Рисунок 2 - Отчёт 1: все рейсы с нумерацией строк

Отчёт 2 - количество рейсов по авиакомпаниям:

```
=== Количество рейсов по авиакомпаниям ===
Авиакомпания            Кол-во рейсов
----------------------------------------
S7 Airlines             3
Utair                   3
Аэрофлот                3
Победа                  3
Уральские авиалинии     2
```

Рисунок 3 - Отчёт 2: количество рейсов

Отчёт 3 - средняя дальность рейсов по авиакомпаниям (по убыванию):

```
=== Средняя дальность рейсов по авиакомпаниям ===
Авиакомпания              Средняя дальность (км)
---------------------------------------------
Аэрофлот                  3914
S7 Airlines               2836,7
Уральские авиалинии       1808
Utair                     1518,3
Победа                    1141
```

Рисунок 4 - Отчёт 3: средняя дальность по убыванию

Вывод: В результате выполнения домашнего задания было разработано консольное приложение для управления авиарейсами с применением принципов ООП. Реализованы классы-модели с инкапсуляцией и валидацией данных, класс DatabaseManager для работы с базой данных SQLite, а также класс ReportBuilder с паттерном Fluent Interface для формирования отчётов. Выполнено задание группы А: добавлен метод Numbered(), позволяющий включать нумерацию строк в любой отчёт через цепочку вызовов.